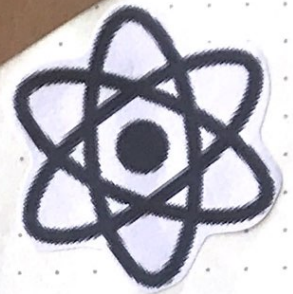


React



¿qué es?

Una biblioteca de JavaScript para construir interfaces de usuario.

```
ReactDOM.render(  
  <h1>Hola, mundo! </h1>  
  document.getElementById('root')  
)
```

* ESTE PEQUEÑO EJEMPLO MUESTRA UN ENCABEZADO CON EL TEXTO:

Hola, mundo!



extensión de la sintaxis de JavaScript.
para uso con **React**.

JSX produce "elementos" de **React**.

- ELEMENTO: Bloque más pequeño de las aplicaciones de **React**.
- Describe lo que quieres ver en la pantalla.

Considerando las siguientes variables:

```
const name = 'Elon Musk';
```

```
const element = <h1> Hello, {name} </h1> ;
```

• Para renderizarlo:

SE PASAN
AMBOS A RENDER

```
ReactDOM.render (
```

```
  element,  
  document.getElementById('root')
```

```
);
```

Imagina que hay un `<div>`
en alguna parte de tu HTML.
`<div id='root'></div>`,
siendo un nodo raíz.

> muestra: **Hello, Elon Musk.**

Se puede utilizar **Jsx** dentro de declaraciones **IF** y bucles **FOR**, asignarlo a variables, aceptarlo como argumento y retomararlo desde dentro de funciones.

atributos con JSX.

- Puedes utilizar comillas para especificar strings como atributos:

```
const element = <div tabIndex="0"></div>;
```

- También usar llaves para insertar una expresión JS:

```
const element = <img src={user.avatarURL}></img>;
```

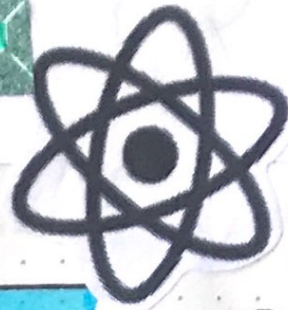
⚠ No utilices comillas rodeando llaves.

"comillas": valores de los strings

{llaves}: expresiones JavaScript

- Si una etiqueta está vacía, puedes cerrarla con `/>`:

```
const element = <img src={user.avatarURL} />
```



JSX

representa objetos



llamados "ELEMENTOS DE REACT"

React lee estos objetos y los usa para construir el DOM

éstos ejemplos son idénticos:

```
const element = (  
  <h1 className = 'greeting'>  
    Hola, mundo!  
  </h1>  
);
```

```
const element = React.createElement(  
  'h1',  
  { className: 'greeting' },  
  'Hola, mundo!'  
);
```

renderizando elementos

"ELEMENTO" } Bloque más pequeño de las aplicaciones de React.

Describe lo que quieres ver en la pantalla.

```
const element = <h1>Hello, world</h1>;
```

Imagina que hay un `<div>` en alguna parte del HTML

```
<div id="root"></div>
```

Para renderizar un elemento de React en un nodo raíz del DOM, pasa ambos a `ReactDOM.render()`

```
const element = <h1>Hello, world</h1>;
```

```
ReactDOM.render(element, document.getElementById('root'));
```

ACTUALIZANDO EL ELEMENTO:

React solo actualiza lo que es necesario.

ReactDOM compara el elemento y sus hijos con el elemento anterior, y solo aplica las actualizaciones del DOM que son necesarias para que el DOM esté en el estado deseado.

:: COMPONENTES ::

permiten separar la interfaz de usuario en piezas independientes.

Los componentes aceptan entradas arbitrarias, llamadas "props", y devuelven a React elementos que describen lo que se debe ver en la pantalla.

La Forma más sencilla de definir un componente es escribir una función de JavaScript



```
function Welcome (props) {  
  return <h1> Hello, {props.name}?  
    </h1>;  
}
```

ACEPTA UN ARGUMENTO DE OBJETO 'PROPS' (que proviene de propiedades) CON DATOS Y DEVUELVE UN ELEMENTO DE REACT.

Estos componentes son "Funcionales" porque literalmente son funciones de JavaScript

renderizando componentes

Además de etiquetas del DOM, los elementos también pueden representar **componentes** definidos por el usuario:

```
function Welcome (props) {  
  return <h1> Hello, { props.name } </h1> ;  
}  
  
const element = <Welcome name = 'Elon' />  
  
ReactDOM.render(  
  element,  
  document.getElementById('root') );
```

- 1: Llamamos a `ReactDOM.render()` con el elemento `<Welcome name = "Elon" />`
- 2: React llama al componente `Welcome` con `{name: 'Elon'}` como `'props'`.
- 3: El componente `Welcome` devuelve un elemento `<h1> Hello, Elon </h1>` como resultado
- 4: React DOM actualiza el DOM para que coincida con `<h1> Hello, Elon </h1>`

react.component.

React te permite definir componentes como **funciones** o **clases**.

Para definir una clase de componente React, necesitas extender **React.Component**:

```
class Welcome extends React.Component {  
  render() {  
    return <h1>Hello, {this.props.name}</h1>  
  }  
}
```

Si aún no utilizas ES6, puedes utilizar el módulo

create-react-class:



```
var createReactClass = require('create-react-class');  
var Greeting = createReactClass({  
  render: function() {  
    return <h1>Hello, {this.props.name}</h1>  
  }  
});
```

métodos de ciclo de vida

- Comúnmente usados -

montaje

Estos métodos se llaman cuando se crea una instancia de un componente y se inserta en el DOM.

• constructor():

SI NO INICIALIZAS EL ESTADO Y NO ENLAZAS LOS MÉTODOS, NO NECITAS IMPLEMENTAR UN CONSTRUCTOR PARA TU COMPONENTE REACT.

Normalmente, se utiliza para dos propósitos:

- Para inicializar un estado local asignando un objeto al `this.state`
- Para enlazar manejadores de eventos a una instancia

• render():

ES EL ÚNICO MÉTODO REQUERIDO EN UN COMPONENTE DE CLASE.

Cuando se llama, examina a `this.props` y `this.state`, y devuelve uno de los siguientes tipos:

- Elementos de React (Ej: `<div />`)
- Arrays y fragmentos: Permiten que puedas devolver múltiples elementos desde el render.
- Portales: Permiten renderizar hijos a otro subárbol del DOM.
- String y números
- Booleanos o nulos: No renderizan nada.

- `componentDidMount()`

Se invoca inmediatamente después de que un componente se monte (se inserte en el árbol).

Este método es un buen lugar para establecer cualquier suscripción. Si lo haces, no olvides darle de baja en `componentWillUnmount()`.

Actualización

Puede ser causada por cambios en los props o el estado.

- `render()`

- `componentDidUpdate()`:

`componentDidUpdate(prevProps, prevState, snapshot)`

Se invoca inmediatamente después de que la actualización ocurra.

Este método no es llamado para el renderizador inicial.

Es un buen lugar para hacer solicitudes de red siempre y cuando compare los accesorios actuales con los anteriores.

Desmontaje

- `componentWillUnmount()`

Se invoca inmediatamente antes de desmontar y destruir un componente. Realiza las tareas de limpieza necesarias en este método.

No debes llamar `setState()` en `componentWillUnmount()` porque el componente nunca será vuelto a renderizar.

manejando eventos

- Los eventos de React se nombran usando camelCase, en vez de minúsculas.
- Con JSX pasas una función como el manejador del evento, en vez de un string.

HTML

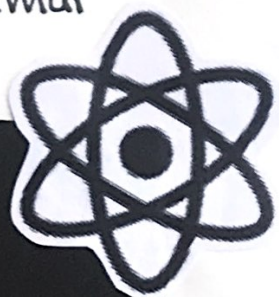
```
<button onclick="activateLasers()">  
  Activate Lasers  
</button>
```

REACT

```
<button onClick={activateLasers}>  
  Activate Lasers  
</button>
```

En React no puedes retornar **false** para prevenir el comportamiento por defecto. Debes llamar **preventDefault**.

```
Function ActionLink() {  
  function handleClick(e) {  
    e.preventDefault();  
    console.log('The link was clicked.');
```



```
  }  
  return (  
    <a href="#" onClick=  
      {handleClick}>  
      Click me </a>  
    )  
  )  
}
```

Los **EVENTOS** en React se definen de manera declarativa, producidos con JSX en el método render().

{—————} Para definirlos {—————}

- Se indica el tipo de evento.

- Indicamos el método que hará de manejador de evento.

💡 un manejador de evento es, básicamente, una función que define la funcionalidad que se ejecutará al dispararse un evento.

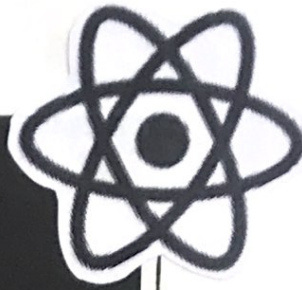
```
render() {  
  return(  
    <p>  
      <button onClick={this.toggleSlider}>Mostrar / Ocultar slider  
    </button>  
    </p>  
  );  
}
```

Las declaraciones de eventos en React tienen siempre la forma:

"on" + tipo de evento (onClick, onChange...)

Luego, definimos el manejador:

```
toggleSlider() {  
  alert('Haz click');  
}
```



Al implementar un manejador de eventos es habitual que queramos acceder a las props o métodos del componente.

Pero en las funciones no es posible acceder a `this` como referencia al objeto sobre el que se invoca el método. Se necesita 'bindear' el contexto:

• Directamente en el constructor del componente

```
constructor(props) {  
  super(props);  
  this.state = {  
    showSlider: false  
  };  
  this.toggleSlider = this.toggleSlider.bind(this);  
}
```

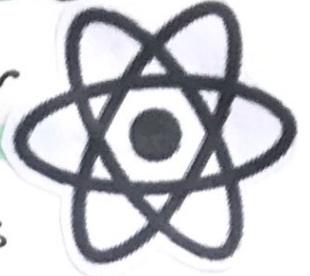
• En el template:

```
<button onClick={this.toggleSlider.bind(this)}>Mostrar / Ocultar Slider </button>
```



renderizando

~ condicional ~



En React, puedes crear distintos componentes que encapsulan el comportamiento que necesitas.

Entonces, puedes renderizar solamente algunos de ellos, dependiendo del estado de tu aplicación.

El renderizado condicional usa operadores de JS como `if` o el `operador condicional` para crear elementos representando el estado actual.

✂ Considera estos dos componentes ✂

```
function UserGreeting(props) {  
  return <h1> Welcome back! </h1>  
}
```

```
function GuestGreeting(props) {  
  return <h1> Please sign up. </h1>  
}
```

✂ Creamos un componente `Greeting` ✂

```
function Greeting(props) {  
  const isLoggedIn = props.isLoggedIn;  
  if (isLoggedIn) {  
    return <UserGreeting />;  
  }  
  return <GuestGreeting />;  
}
```

```
ReactDOM.render(  
  // Intentar cambiando isLoggedIn = {true}:  
  <Greeting isLoggedIn = {false} />;  
  document.getElementById('root')  
);
```

evitar que el componente se renderice

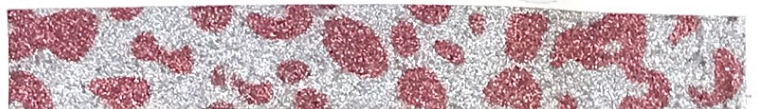
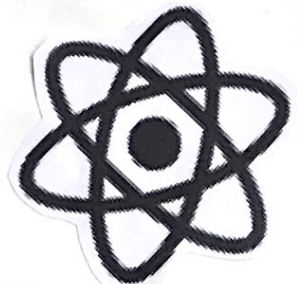
Es posible que desees que un componente se oculte a sí mismo aunque haya sido renderizado por otro componente.

Para hacer esto, devuelve **null**

```
function Warning Banner (props) {  
  if (!props.warn) {  
    return null;  
  }  
  return (  
    <div className = "warning">  
      Warning  
    </div>  
  );  
}
```

*Si el valor del prop es **False**, entonces el componente no se renderiza.

El devolver **null** desde el método **render** de un componente no influye en la activación de los métodos del ciclo de vida del componente. Por ejemplo, **component DidUpdate** seguirá siendo llamado.



listas

RENDERIZADO DE MÚLTIPLES COMPONENTES.

Puedes hacer colecciones de elementos e incluirlos en JSX usando llaves `{}`

```
const numbers = [1, 2, 3, 4, 5];  
const listItems = numbers.map((number) =>  
  <li>{number}</li>  
);
```

recorremos el array `numbers` usando la Funcion `map()` de JavaScript. Devolvemos un elemento `<li>` por cada item. Finalmente, asignamos el array de elementos resultante a `listItems`.

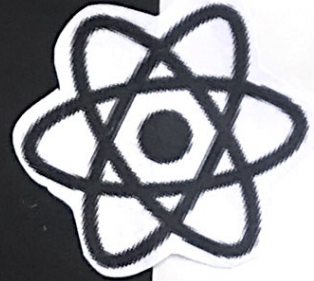
```
ReactDOM.render(  
  <ul> {listItems} </ul>,  
  document.getElementById('root')  
)
```

Incluimos entero el array `listItems` dentro de un elemento `<ul>`, y lo `renderizamos` al DOM.

*Este código muestra una lista de números entre 1 y 5.

Refactorizamos el ejemplo anterior en un componente que acepte un array de **numbers** e imprima una lista de elementos:

```
function NumberList(props) {  
  const numbers = props.numbers;  
  const listItems = numbers.map((number) =>  
    <li> {number} </li>  
  );  
  return (  
    <ul> {listItems} </ul>  
  );  
}  
  
const numbers = [1, 2, 3, 4, 5];  
ReactDOM.render(  
  <NumberList numbers={numbers} />,  
  document.getElementById('root')  
);
```



Al ejecutar este código, serás advertido que una **key** debería ser proporcionada para items de lista.

```
<li key={number.toString()}>  
  {number}  
</li>
```

KEY?

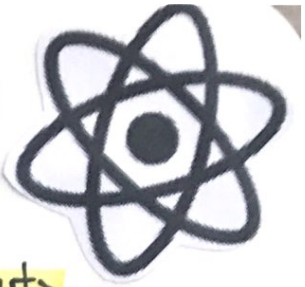
Ayudan a React a identificar que items han cambiado, son agregados, o son eliminados. Las keys deben ser dadas a los elementos dentro del array.

Habitualmente vas a usar IDs de tus datos como key.

```
<li key={todo.id}>  
  {todo.text} </li>
```



F.O.R.M.U.L.A.R.I.O.S



En HTML, los elementos de formularios como los `<input>`, `<textareas>` y el `<select>` mantienen sus propios estados y los actualizan de acuerdo a la interacción del usuario.

En React, el estado mutable es mantenido normalmente en la propiedad del estado de los componentes, y solo se actualiza con `setState()`.

textarea

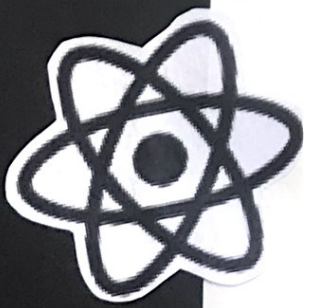
En React, un `<textarea>` utiliza un atributo `value`, y es inicializado en el constructor.

```
class myForm extends React.Component {  
  constructor(props) {  
    super(props);  
    this.state = {  
      description: 'The content of a textarea goes in the  
        value attribute' ?  
    };  
  }  
  render() {  
    return (  
      <Form>  
        <textarea value = {this.state.description} />  
      </Form> );  
    }  
  }  
}
```

select

React utiliza el atributo **value** en la raíz de la etiqueta **select**.

```
class MyForm extends React.Component {
  constructor(props) {
    super(props);
    this.state = {
      mycar: 'Cybertruck'
    };
  }
  render() {
    return (
      <Form>
        <select value={this.state.mycar}>
          <option value="Twingo">Twingo</option>
          <option value="Cybertruck">Cybertruck</option>
        </select>
      </Form> );
  }
}
```



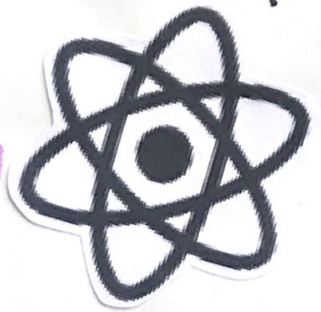
Puedes pasar un array al atributo **value**, permitiendo que selecciones múltiples opciones en una etiqueta **select**:

```
<select multiple={true} value={['b', 'c']}>
```

En resumen, esto hace que `<input type="text">`, `<textarea>` y `<select>` trabajen de manera similar, todos aceptan un atributo `value` el cual puedes usar para implementar un componente controlado.

Un campo de un formulario cuyos valores son controlados por React es denominado 'componente controlado'.

Levantando el estado

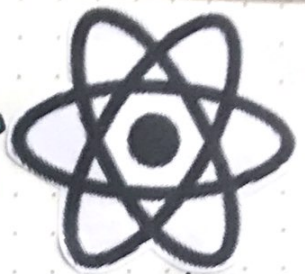


En React, la compartición del estado puede lograrse moviendo el estado hacia arriba al ancestro común más cercano de los componentes que lo necesitan. A esto se le llama "levantar el estado".

El componente que posee el estado compartido se convierte en la "Fuente de verdad".



= composición



{ React tiene un potente modelo de composición, y se recomienda usar composición en lugar de herencia }

~ CONTENCIÓN ~

Algunos componentes no conocen sus hijos de antemano. Es común para componentes como Sidebar o Dialog que representan "cajas" genéricas.

Se recomienda que estos componentes usen la prop especial children para pasar elementos hijos directamente en su resultado:

```
function FancyBorder (props) {  
  return (  
    <div className = { 'Fancy Border FancyBorder-' + props.color } >  
      { props.children }  
    </div>  
  );  
}
```

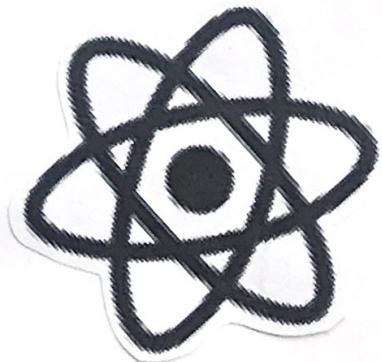
Esto permite que otros componentes les pasen hijos arbitrarios andando el JSX.

```
Function WelcomeDialog () {  
  return (  
    <FancyBorder color="blue">  
      <h1 className="Dialog-title">  
        Welcome  
      </h1>  
      <p className="Dialog-message">  
        Thank you for visiting  
      </p>  
    </FancyBorder> );  
  }  
}
```

∴ ESPECIALIZACIÓN ∴

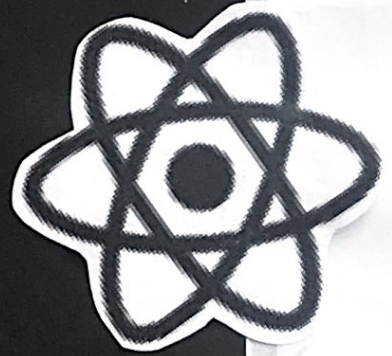
A veces pensamos en componentes como "casos" concretos de otros componentes.

Por ejemplo, podríamos decir que un **WelcomeDialog** es un caso concreto de **Dialog**.



En *react*, esto también se consigue por composición, en la que un componente más "ESPECÍFICO" renderiza uno más "GENÉRICO" y lo configura con **props**:

```
function Dialog(props) {  
  render (  
    <Fancy Border color="blue">  
      <h1 className="Dialog-title">  
        {props.title}  
      </h1>  
      <p className="Dialog-message">  
        {props.message}  
      </p>  
    </Fancy Border>  
  );  
}
```



```
function WelcomeDialog() {  
  return (  
    <Dialog  
      title="welcome"  
      message="Thank you for visiting" />  
  );  
}
```

Fuente

- Documentación oficial de ReactJs:
"conceptos principales"

<https://es.reactjs.org/docs>

- "Eventos":

<https://desarrolloweb.com>

Hecho con ❤ por Majo

Tw: @MajoLedes

Enero, 2020.-

hooks

¿qué es un Hook?

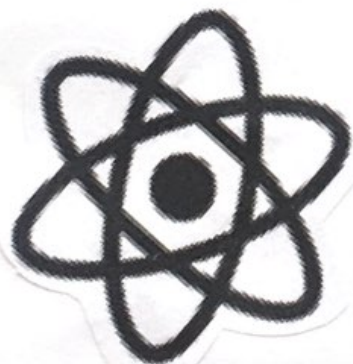
Hooks son Funciones que te permiten "enganchar" el estado de React y el ciclo de vida desde componentes Funcionales.

Los Hooks no funcionan dentro de las clases, te permiten usar React sin clases.

React proporciona algunos Hooks incorporados como `useState`. *

`useEffect` *

`useContext` *



useState

```
const [state, setState] = useState(initialState);
```

{ Devuelve un valor con estado
y una función para actualizarlo. }

- Durante el renderizado inicial, el estado devuelto (`state`) es el mismo que el valor pasado como primer argumento (`initialState`)
- La función `setState` se usa para actualizar el estado. Acepta un nuevo valor de estado y sitúa en la cola una nueva renderización del componente.

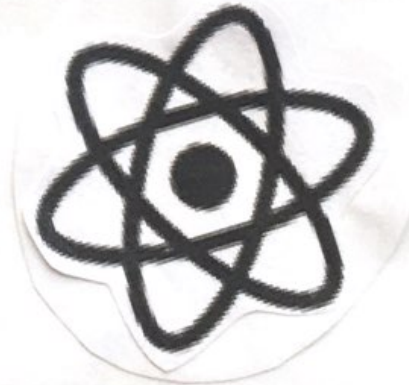
```
setState(newState);
```

En las renderizaciones siguientes, el primer valor devuelto por `useState` será siempre el estado más reciente después de aplicar las actualizaciones.

- ⚠ El argumento `initialState` es el estado utilizado durante el render inicial. En renderizados posteriores, se ignora.

CON CLASES

```
class Contador extends React.Component {  
  constructor(props) {  
    super(props);  
    this.state = {  
      count: 0  
    };  
  }  
  
  render() {  
    return (  
      <div>  
        <p> Haz clic {this.state.count} veces </p>  
        <button onClick={() => this.setState({ count :  
          this.state.count + 1 })} > Haz clic!  
        </button>  
      </div>  
    );  
  }  
}
```



con hooks 😊

```
import React, {useState} from 'react'
```

```
function Contador () {
```

```
  const [count, setCount] = useState(0)
```

```
  <div>
```

```
    <p> Hazle click {count} VECES </p>
```

```
    <button onClick={() => setCount(count + 1)}> Haz click!
```

```
  </button>
```

```
  </div>
```

```
};
```

```
?
```

```
5
```



- Importamos **useState** que nos permite mantener un Estado local en un Componente Funcional.
- Dentro del componente **Contador** declaramos la variable **count** (guarda el número de clicks) llamando al hook **useState**. La inicializamos en cero pasando (0) como único argumento a **useState**.
- **SetCount** nos permite actualizar el **count**.
- Cuando el usuario hace click, llamamos a **setCount** con un nuevo valor.
- **React** actualiza el componente **Contador** pasándole el nuevo valor de **count**.

useEffect

Al usar este Hook, le estamos indicando a React que el componente tiene que hacer algo después de renderizarse. React recordará la función que le hemos pasado (nos referimos a ella como nuestro "Efecto") y la llamará más tarde después de actualizar el DOM.

USEEFFECT equivale a la combinación de :

- * component Did Mount,
- * component Did Update,
- * component Will Unmount.



con classes



```
class FriendStatusWithCounter extends React.Component {  
  constructor(props) {  
    super(props)  
    this.state = { isOnline: null };  
    this.handleStatusChange = this.handleStatusChange.bind(this);
```

```
  componentDidMount() {  
    document.title = `Clickeaste ${this.state.count} VECES`; ;  
    chatAPI.subscribeToFriendStatus(  
      this.props.friend.id,  
      this.handleStatusChange);  
  }
```

```
  componentDidUpdate() {  
    document.title = `Clickeaste ${this.state.count} VECES`; ;
```

```
  componentWillUnmount() {  
    ChatAPI.unsubscribeFromFriendStatus(  
      this.props.friend.id,  
      this.handleStatusChange);  
  }
```

```
  handleStatusChange(status) {  
    this.setState({  
      isOnline: status.isOnline; })
```

```
  render() {  
    if (this.state.isOnline === null) {  
      return 'Loading...';  
    }  
    return this.state.isOnline ? 'Online' : 'Offline';  
  }
```

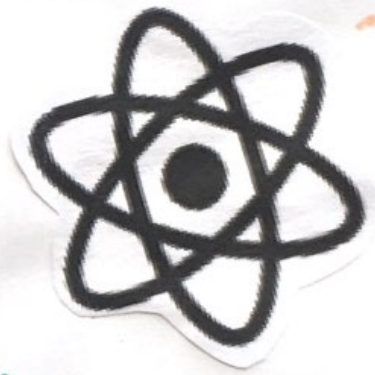
En el ejemplo anterior se añadió una funcionalidad: Actualizar el título del documento con un mensaje personalizado que incluye el número de clicks.

Esta lógica se divide entre `componentDidMount` y `componentDidUpdate`.

La lógica de la suscripción (tenemos un módulo ChatAPI que nos permite suscribirnos para saber si un amigo está conectado) se reparte entre `componentDidMount` y `componentWillUnmount`.

SE SUSCRIBE

SE CANCELA LA
SUSCRIPCIÓN



con hooks



¡PUEDES USAR EL HOOK DE EFECTO MÁS DE UNA VEZ!

Esto nos permite separar la lógica que no está relacionada en diferentes efectos

```
function FriendStatusWithCounter(props) {  
  const [count, setCount] = useState(0);  
  useEffect(() => {  
    document.title = `Clickeaste ${count} veces`;  
  });  
  const [isOnline, setIsOnline] = useState(null);  
  useEffect(() => {  
    function handleStatusChange(status) {  
      setIsOnline(status.isOnline);  
    }  
    ChatAPI.subscribeToFriendStatus(props.friend.id,  
    handleStatusChange);  
    return () => {  
      ChatAPI.unsubscribeToFriendStatus(props.friend.id,  
      handleStatusChange);  
    };  
  });  
  //...  
}
```

useContext



```
const value = useContext(MyContext);
```

ACEPTA UN OBJETO DE CONTEXTO (el valor devuelto de `React.createContext`) Y DEVUELVE EL VALOR DE CONTEXTO ACTUAL.

EL VALOR ACTUAL DEL CONTEXTO ES DETERMINADO POR LA PROPIEDAD `value` DEL `<MyContext.Provider>` ASCENDENTEMENTE MÁS CERCANO EN EL ÁRBOL AL COMPONENTE QUE HACE LA LLAMADA.

Cuando el `<MyContext.Provider>` ASCENDENTEMENTE MÁS CERCANO EN EL ÁRBOL SE ACTUALIZA, EL HOOK ACTIVA UNA RENDERIZACIÓN CON EL `value` MÁS RECIENTE DEL CONTEXTO PASADO A ESE PROVEEDOR `<MyContext>`

¡No olvides que el argumento enviado a `useContext` debe ser el objeto del contexto en sí mismo:

```
useContext(MyContext) !
```



Hooks adicionales

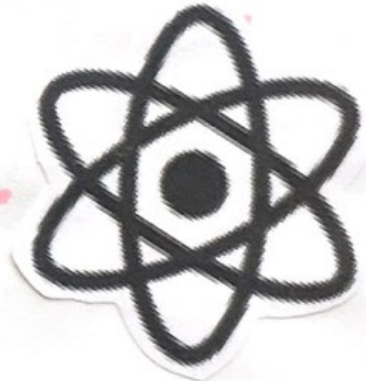
useReducer:

```
const [state, dispatch] = useReducer(reducer, initialArg, init);
```

Es una alternativa a `useState`. Acepta un reducer de tipo `(state, action) => newState` y devuelve el estado actual emparejado con un método `dispatch`.

useCallback:

```
const memoizedCallback = useCallback(  
  () => {  
    doSomething(a, b);  
  },  
  [a, b],  
);
```



Pasa un callback en línea y un arreglo de dependencias. `useCallback` devolverá una versión memorizada del callback que solo cambia si una de las dependencias ha cambiado.

useMemo:

```
const memoizedValue = useMemo(() => computeExpensiveValue(a, b),  
[a, b]);
```

Devuelve un valor memorizado.

Pasa una función de "crear" y un arreglo de dependencias.

useMemo sólo volverá a calcular el valor memorizado cuando una de las dependencias haya cambiado.

La función **useMemo** se ejecuta durante el renderizado.

useRef:

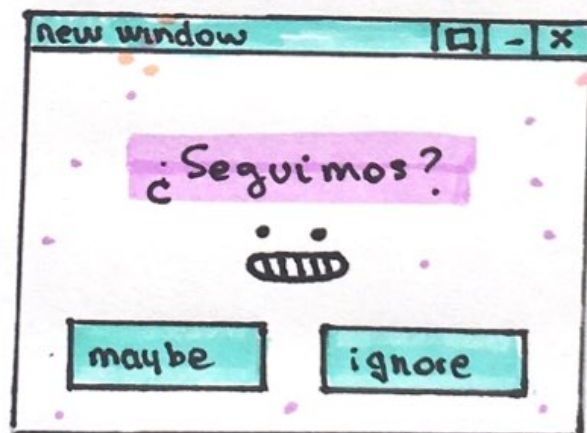
```
const refContainer = useRef(initialValue);
```

useRef devuelve un objeto ref mutable cuya propiedad

current se inicializa con el argumento pasado

(**initialValue**). El objeto devuelto se mantendrá

persistente durante la vida completa del componente.



reglas de los hooks



Los hooks son FUNCIONES de JavaScript, pero NECESITAS seguir dos reglas cuando los uses.

- Llama a los Hooks solo EN EL NIVEL SUPERIOR



→ No llames Hooks dentro de ciclos, condicionales o funciones anidadas.

siguiendo esta regla, te aseguras de que los hooks se llamen en el mismo orden cada vez que un componente se renderiza.

- Llama Hooks solo EN FUNCIONES DE React.



→ No llames Hooks desde funciones JavaScript regulares.

siguiendo esta regla, te aseguras de que toda la lógica del Estado de un componente sea claramente visible desde tu código fuente.

Hay un plugin de ESLint llamado `eslint-plugin-react-hooks` que refuerza estas dos reglas.

Este plugin es incluido por defecto en `Create React App`.